Ускорение инференса UNet

для задачи Super-Resolution x 4

Efficient ML — Team 52

Артём Чубов · Женя Веселков · Андрей Петухов

Апрель 2026

Задача: Super-Resolution x 4

LR вход (180x 140 px)

SR выход (720x 560 px)

Что делает модель

По Low-Resolution входу восстанавливает детали — масштаб x 4

LR 256x 256 → SR 256x 256

(вычисления на LR-разрешении)

Датасет: DIV2K

2K HR-фотографии природы

Train	800 изображений
Val	50 изображений
Test	50 изображений

Модель: SRUNetHeavy

Архитектура

- U-Net, 5 уровней энкодер/декодер
- **MBResBlock** (inverted residual): Conv1x 1 → DWConv3x 3 → Conv1x 1
- Глобальный residual
- **GroupNorm** — каналы можно удалять без пересчёта статистик

Ключевые параметры

Параметры	**29.3M**
FLOPs	138 GFLOPs
Вход/выход	256x 256 → 256x 256
Метрика	SSIM (FP32 ref = **0.7689**)
Бенчмарк	CUDA Events

Метрика ускорения

Speedup = *baseline latency / result latency*

Контроль качества: SSIM vs FP32 baseline

Главные результаты — все методы

Метод	Precision	Speedup	SSIM	GPU
Eager baseline	FP32	1.00x	0.769	RTX 3060
Eager baseline	FP16	1.52x	0.769	RTX 3060
torch.compile	FP32	1.03x	0.769	RTX 3060
ORT TensorRT EP	FP32	1.61x	0.769	RTX 3060
ORT TRT EP	**FP16**	**3.62x**	**0.769**	RTX 3060
LSQ PTQ + FP16	INT8/FP16	≈1.0x *	0.768	A100
Struct. pruning 25%	FP32	1.23x	0.760	RTX 8000
Struct. pruning 50%	FP32	1.57x	0.758	RTX 8000
Struct. pruning 75%	**FP32**	**2.27x**	**0.760**	RTX 8000
2:4 sparsity (FP16)	FP16	0.84x	0.75	A100

* LSQ PTQ: главный выигрыш — сжатие модели в **4x** (112 MB → 28 MB)

Квантизация LSQ — метод

LSQ: Learned Step Size Quantization

Fake quantization — форвард симулирует INT8,
веса остаются в float:

$$\hat{w} = \text{round}(w \,/\, s) \cdot s$$

- s — learnable per-channel step size

Сжатие модели

Формат	Размер	SSIM
FP32	111.7 MB	0.7689
FP16	55.8 MB	0.7689
LSQ INT8	**28.1 MB**	**0.7680**

4x меньше FP32
SSIM $\Delta = -0.0009$ (без finetune)

Квантизация LSQ — результаты

NVIDIA A100-SXM4-80GB · 256x 256 · 100 runs

Config	BS=1	BS=4	BS=16	BS=64
FP32	30.2 ms	51.0 ms	128.9 ms	485.8 ms
FP16	26.1 ms	41.0 ms	89.3 ms	306.9 ms
LSQ+FP32	41.3 ms	61.5 ms	139.5 ms	496.0 ms
LSQ+FP16	37.7 ms	51.9 ms	99.9 ms	**316.8 ms**

FP32 111.7 MB
FP16 55.8 MB
INT8 28.1 MB

Выводы

- LSQ+FP16 при BS=64 → **0.97x** vs FP16
- Выигрыш: **хранение** (4x)
- Для speedup нужны INT8-ядра (TRT INT8, torch.ao)

Компиляция

Метод	Prec.	Latency	Speedup
Eager PyTorch	FP32	48.1 ms	1.00x
Eager PyTorch	FP16	31.7 ms	1.52x
torch.compile	FP32	46.6 ms	1.03x
ORT CUDA EP	FP32	46.1 ms	1.04x
ORT TRT EP	FP32	29.8 ms	1.61x
ORT TRT EP	**FP16**	**13.3 ms**	**3.62x**

SRUNetHeavy · RTX 3060

TRT FP16: почему 3.62x?

- **Fusion**: TRT склеивает несколько операций в один kernel.
- **FP16**: вдвое меньше байт на каждое чтение/запись

TVM MetaSchedule (UNetSR)

Автотюнинг CUDA-тайлингов без cuDNN.

time spent	Latency	Speedup
(default)	511 ms	0.23x
3 hours	136 ms	0.88x

Близко к cuDNN, но не обгоняет.

Компиляция

Arithmetic Intensity

	AI (FL/B)	Ridge	Режим
FP32	24	35.4	**mem-bound**
FP16	48	70.8	**mem-bound**

Утилизация bandwidth

Метод	**% от BW ceiling**
Eager FP32	33%
TRT FP32	54%
TRT FP16	**60%**

Потолок: 360 GB/s $\times$ AI

Прунинг

Channel Pruning

В каждом MBResBlock физически удаляем каналы 1x1 свёртки с наименьшей L1-нормой

Архитектура не нарушается, только кол-во промежуточных каналов уменьшается

Почему даёт ускорение

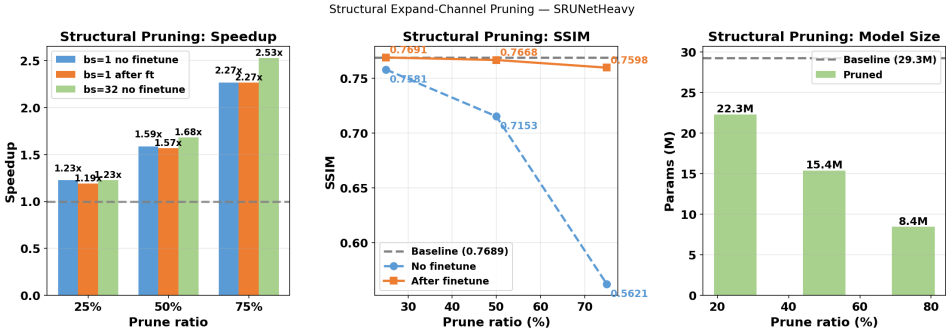
- Тензоры меньше физически — не нужны sparse kernels
- GroupNorm — каналы удаляются без пересчёта
- Finetune (10 эпох) восстанавливает SSIM

Результаты: 75% + finetune

	До	После
Параметры	29.3M	**8.4M**
Latency	34.2 ms	**15.1 ms**
Speedup	—	**2.27x**
SSIM	0.7689	0.7598

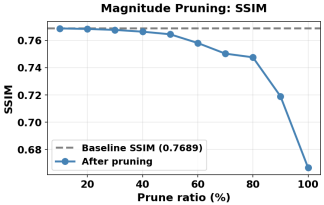
Модель в **3.5x** меньше · SSIM **−0.009**

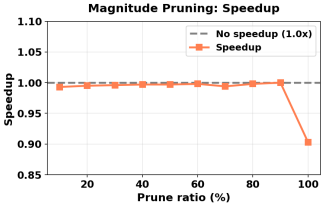
Прунинг - результаты



Прунинг	Speedup BS=1	Speedup BS=32	SSIM (ft)	Params
25% + finetune	1.23x	1.23x	0.760	22.3M
50% + finetune	1.57x	1.68x	0.758	15.4M
75% + finetune	**2.27x**	**2.53x**	**0.760**	**8.4M**

Magnitude Pruning





Метод

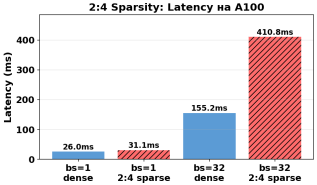
Обнуляем N% наименьших весов 1x 1 свёрток

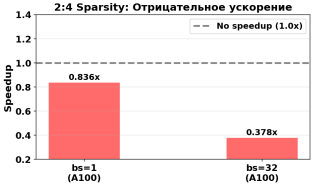
Ускорения примерно нет

Тензоры остаются плотными — GPU исполняет те же CUDA-ядра с теми же матрицами

Нули перемножаются, а не пропускаются

2:4 Semi-structured Sparsity





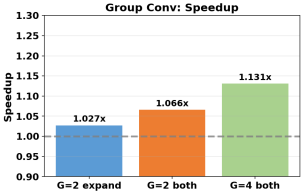
Метод

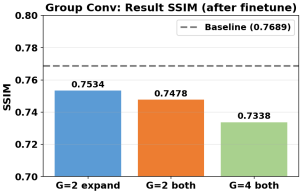
В каждой группе из 4 весов ровно 2 обнуляются (по наименьшей норме).

Почему нет ускорения

- Нужен сильно больше батчсайз + конвертировать свёртку в линейный слой
- На A100: матрицы 1x 1 свёрток слишком малы, чтобы увидеть ускорение
- BS=1: **0.84x**, BS=32: **0.38x**

Grouped Convolutions





Метод

expand/project 1x 1 Conv → grouped conv (G=2 или G=4).

GFLOPs, но CUDA matmul overhead ограничивает реальный speedup.

Сравнение

G=4 both: **1.13x**, SSIM -0.035

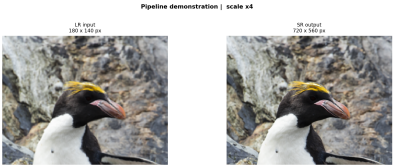
Struct. 75%: **2.27x**, SSIM -0.009

→ Структурированный прунинг превосходит по обеим метрикам

Выводы

Лучшие результаты

Цель	Метод	Итог
Макс. ускорение	TRT FP16	**3.62x**
Без потерь SSIM	TRT FP16	SSIM = 0.769
Сжатие модели	LSQ PTQ	**4x**
Speed+компактность	Struct. 75%	**2.27x**



Ключевые наблюдения

- **torch.compile** — минимальный выигрыш:
  Triton не фьюзит GroupNorm+GELU+DWConv
- **TensorRT** — лучший бэкенд: fusion + Tensor Cores
- **LSQ PTQ** — 4x меньше, но без INT8-ядер не ускоряет
- **Struct. pruning** — лучший трейдофф, любой GPU
- **2:4 sparsity** — нужен Ampere+, малые матрицы убыточны